

NiceGUI ui.query 完全指南（基于官方文档深度解析）

`ui.query` 是 NiceGUI 提供的强大 DOM 查询与操作工具，灵感源自 jQuery，允许通过 CSS 选择器定位 UI 元素，并批量执行样式修改、属性设置、事件绑定等操作。其核心价值在于**跨组件层级**、**批量处理元素**，尤其适合动态 UI 场景（如主题切换、批量禁用 / 启用、全局样式调整）。

一、核心定义与设计理念

1. 本质与定位

- `ui.query(selector: str) -> QueryResult`: 接收 CSS 选择器字符串，返回 `QueryResult` 对象（包含匹配的元素集合）。
- 设计目标：简化多元素操作逻辑，避免手动遍历组件引用，实现「选择器定位 + 链式操作」的高效开发模式。
- 适用场景：全局样式统一修改、批量状态切换（禁用 / 隐藏）、动态 UI 调整（如响应式布局）、跨组件层级操作（无需传递组件引用）。

2. 与直接操作组件的区别

特性	<code>ui.query</code> 方式	直接操作组件方式
操作范围	批量匹配符合选择器的所有元素	单个组件或已知引用的多个组件
组件引用依赖	无需持有组件引用，通过选择器定位	必须持有组件实例引用（如变量赋值）
代码简洁度	一行代码完成批量操作，链式调用更优雅	需循环遍历组件列表，代码冗余
动态性	支持动态新增元素（后续匹配选择器）	新增元素需手动添加到操作列表
适用场景	全局样式调整、批量状态切换、跨层级操作	单个组件精细化控制、局部逻辑处理

二、基础用法：选择器语法（CSS Selector 兼容）

`ui.query` 完全支持 CSS 选择器语法，可通过元素类型、类名、ID、属性等定位元素，以下是 NiceGUI 开发中高频使用的选择器场景：

选择器类型	语法示例	说明（匹配目标）
元素类型（标签名）	<code>ui.query('button')</code>	所有按钮元素（HTML 标签为 <code><button></code> ）
类名（ <code>classes</code> ）	<code>ui.query('.my-custom-btn')</code>	所有绑定 <code>my-custom-btn</code> 类的元素
ID（ <code>id</code> 属性）	<code>ui.query('#user-input')</code>	ID 为 <code>user-input</code> 的元素（需通过 <code>attrs</code> 设置 ID）
组合选择器	<code>ui.query('input.my-input')</code>	所有绑定 <code>my-input</code> 类的输入框元素

选择器类型	语法示例	说明（匹配目标）
层级选择器	<code>ui.query('.card .label')</code>	所有 <code>card</code> 类元素下的 <code>label</code> 子元素
属性选择器	<code>ui.query('[placeholder*="用户名"]')</code>	占位符包含「用户名」的输入框
伪类选择器	<code>ui.query('button:not(.disabled)')</code>	所有未绑定 <code>disabled</code> 类的按钮

选择器示例代码

```
from nicegui import ui

# 1. 按元素类型选择（所有按钮）
ui.button("按钮1")
ui.button("按钮2")
ui.query("button").classes(add="bg-blue-500 text-white") # 批量设置按钮样式

# 2. 按类名选择（所有 .item 元素）
with ui.row():
    ui.label("项目1").classes("item")
    ui.label("项目2").classes("item")
ui.query(".item").classes(add="text-gray-700 p-2") # 批量设置项目样式

# 3. 按 ID 选择（单个元素）
ui.input(attrs={"id": "user-input"}).placeholder("请输入用户名")
ui.query("#user-input").classes(add="w-64 border-gray-300") # 定位单个输入框

# 4. 组合选择器（带特定类的输入框）
ui.input(classes="form-control").placeholder("输入框1")
ui.input(classes="form-control").placeholder("输入框2")
ui.query("input.form-control").classes(add="rounded-md px-4") # 批量设置表单输入框样式

ui.run()
```

三、核心操作：QueryResult 链式方法

`ui.query` 返回的 `QueryResult` 对象支持链式调用，以下是官方文档重点强调的核心操作方法，按使用频率排序：

1. 样式操作（classes 相关）

- `classes(add: str = None, remove: str = None) -> QueryResult`：批量添加 / 移除类名（与元素的 `classes` 方法用法一致）。

```
# 批量修改按钮样式：添加绿色背景，移除默认内边距
ui.query("button").classes(add="bg-green-500", remove="px-4 py-2")
```

2. 属性操作 (attrs 相关)

- `attrs(add: dict = None, remove: list = None) -> QueryResult`: 批量添加 / 移除 HTML 属性。

```
# 批量设置输入框为只读, 移除 disabled 属性
ui.query("input").attrs(add={"readonly": True}, remove=["disabled"])
```

3. 内容操作

- `text(value: str) -> QueryResult`: 批量设置元素文本内容 (适用于 `label`、`button` 等文本元素)。

```
# 批量更新所有 .status 标签的文本
ui.query(".status").text("已完成")
```

- `set_text(value: str) -> QueryResult`: 与 `text()` 功能一致, 语义更明确。

4. 状态操作

- `disable() -> QueryResult`: 批量禁用元素 (添加 `disabled` 类和属性)。
- `enable() -> QueryResult`: 批量启用元素 (移除 `disabled` 类和属性)。
- `show() -> QueryResult`: 批量显示元素 (设置 `display: block`)。
- `hide() -> QueryResult`: 批量隐藏元素 (设置 `display: none`)。

```
# 批量禁用所有按钮, 隐藏所有 .hint 元素
ui.query("button").disable()
ui.query(".hint").hide()
```

5. 样式直接设置 (style)

- `style(style: str) -> QueryResult`: 批量设置行内样式 (优先级高于 CSS 类)。

```
# 批量设置元素行内样式: 字体大小 16px, 颜色红色
ui.query(".warning").style("font-size: 16px; color: red;")
```

6. 事件绑定

- `on(event: str, handler: Callable) -> QueryResult`: 为所有匹配元素绑定同一事件处理器。

```
# 为所有按钮绑定点击事件
def on_button_click():
    ui.notify("按钮被点击")

ui.query("button").on("click", on_button_click)
```

7. 元素遍历

- `each(handler: Callable[[Element], None]) -> QueryResult`: 遍历所有匹配元素，执行自定义逻辑（传入元素实例）。

```
# 遍历所有输入框，打印其占位符
ui.query("input").each(lambda e: print(f"占位符: {e.placeholder}"))
```

8. 其他实用方法

方法名	示例代码	说明
<code>count()</code>	<code>ui.query("button").count()</code>	返回匹配元素的数量
<code>get(index: int)</code>	<code>ui.query("button").get(0)</code>	获取索引为 0 的元素实例（单个元素操作）
<code>first()</code>	<code>ui.query("button").first()</code>	获取第一个匹配元素
<code>last()</code>	<code>ui.query("button").last()</code>	获取最后一个匹配元素
<code>parent()</code>	<code>ui.query(".item").parent()</code>	匹配所有元素的父元素
<code>children()</code>	<code>ui.query(".card").children()</code>	匹配所有元素的直接子元素

四、高级场景：动态 UI 与批量操作示例

1. 主题切换（全局样式批量修改）

```
from nicegui import ui

def toggle_dark_mode():
    if dark_mode.value:
        # 深色模式：批量修改背景、文本颜色
        ui.query("body").classes(add="bg-gray-900 text-white")
        ui.query("button").classes(add="bg-gray-700 hover:bg-gray-600")
        ui.query("input").classes(add="bg-gray-800 border-gray-700 text-white")
    else:
        # 浅色模式：移除深色类
        ui.query("body").classes(remove="bg-gray-900 text-white")
        ui.query("button").classes(remove="bg-gray-700 hover:bg-gray-600")
        ui.query("input").classes(remove="bg-gray-800 border-gray-700 text-white")

dark_mode = ui.reactive(False)
ui.switch("深色模式", value=dark_mode, on_change=toggle_dark_mode)

# 测试元素
ui.input("用户名").placeholder("请输入用户名")
ui.button("提交")
ui.card().add(ui.label("卡片内容"))

ui.run()
```

2. 表单批量验证与状态切换

```
from nicegui import ui

def validate_form():
    # 重置所有输入框样式
    ui.query("input.form-field").classes(remove="border-red-500")
    # 模拟验证逻辑: 检查空值
    empty_fields = []
    ui.query("input.form-field").each(lambda e: empty_fields.append(e) if not e.value else None)
    if empty_fields:
        # 批量标记空输入框为红色边框
        for field in empty_fields:
            field.classes(add="border-red-500")
        ui.notify("请填写所有必填字段", type="error")
    else:
        ui.notify("表单验证通过", type="success")

# 表单元素 (统一添加 form-field 类)
ui.input("用户名", classes="form-field").required()
ui.input("密码", classes="form-field").password(True).required()
ui.button("提交", on_click=validate_form)

ui.run()
```

3. 动态新增元素后的批量操作

```
from nicegui import ui

# 初始元素
ui.button("初始按钮", classes="dynamic-btn")

# 动态添加按钮
def add_button():
    ui.button("动态按钮", classes="dynamic-btn")

ui.button("添加按钮", on_click=add_button)

# 批量修改所有 .dynamic-btn 样式 (包括后续动态添加的)
def update_button_style():
    ui.query(".dynamic-btn").classes(add="bg-purple-500 text-white rounded-lg p-2")

ui.button("更新所有按钮样式", on_click=update_button_style)

ui.run()
```

4. 跨组件层级操作（无需传递引用）

```
from nicegui import ui

def toggle_card_content():
    # 定位 card 组件下的所有 .content 元素，切换显示/隐藏
    content_elements = ui.query(".card .content")
    if content_elements.count() > 0:
        if content_elements.first().props.get("hidden", False):
            content_elements.props(remove=["hidden"])
        else:
            content_elements.props(add={"hidden": True})

# 嵌套组件结构
with ui.card(classes="card"):
    ui.label("卡片标题").classes("font-bold")
    ui.text("卡片内容1").classes("content")
    ui.text("卡片内容2").classes("content")

with ui.card(classes="card"):
    ui.label("另一张卡片").classes("font-bold")
    ui.text("卡片内容3").classes("content")

ui.button("切换所有卡片内容", on_click=toggle_card_content)

ui.run()
```

五、关键注意事项（基于官方文档提醒）

1. 选择器匹配时机

- `ui.query` 是**实时查询**：每次调用都会重新扫描当前 DOM 树，匹配符合选择器的元素（包括动态新增的元素）。
- 若需多次操作同一组元素，可缓存查询结果：

```
buttons = ui.query("button") # 缓存查询结果
buttons.classes(add="bg-blue-500")
buttons.disable() # 复用结果，避免重复查询
```

2. 优先级与样式冲突

- 行内样式（`style()` 方法）优先级高于 CSS 类（`classes()` 方法），若需覆盖行内样式，需使用 `!important`：

```
ui.query("button").style("background-color: red !important") # 强制覆盖行内样式
```

- 与 NiceGUI 内置 CSS Layers 兼容：自定义样式需遵循 Layer 优先级规则（参考前文 `ui.add_css` 章节）。

3. ID 唯一性

- 通过 `attrs={ "id": "xxx" }` 设置的 ID 必须唯一，否则 `ui.query("#xxx")` 只会匹配第一个元素。
- 推荐优先使用类名（`classes`）进行批量选择，ID 仅用于单个元素定位。

4. 性能考虑

- 避免频繁调用复杂选择器（如层级过深、属性匹配），尤其是在元素数量较多的场景（会遍历整个 DOM 树）。
- 批量操作优先使用 `ui.query` 而非循环遍历组件引用（`ui.query` 内部优化了 DOM 操作效率）。

5. 元素类型映射

- NiceGUI 组件与 HTML 标签的映射关系（部分）：
 - `ui.button` → `<button>`
 - `ui.input` → `<input>`
 - `ui.label` → `<span>`
 - `ui.card` → `<div class="card">`
 - `ui.row` → `<div class="row">`
- 选择器需基于 HTML 标签名，而非 NiceGUI 组件名（如 `ui.query("span")` 匹配 `ui.label`）。

六、官方文档补充说明

1. 与 `ui.add_css` 的配合：`ui.query` 适合动态修改元素样式 / 状态，`ui.add_css` 适合静态全局样式定义，两者可结合使用：

```
# 静态定义样式类
ui.add_css(".highlight { border: 2px solid yellow; }")
# 动态为元素添加类
ui.query("input:focus").classes(add="highlight")
```

2. 响应式变量结合：可通过 `ui.reactive` 变量触发 `ui.query` 操作，实现状态与 UI 的联动（如前文主题切换示例）。
3. 限制场景：
 - 部分 NiceGUI 组件（如 `ui.table`、`ui.chart`）内部结构复杂，直接通过选择器操作子元素可能不稳定（建议优先使用组件自身的 API）。
 - 若需精细化控制单个元素，建议直接持有组件引用，而非依赖 `ui.query`。